OPERATING SYSTEMS · INTERVIEW NOTES

Operating Systems *Complete* Interview Notes

Processes, threads, scheduling, synchronisation, deadlocks, paging and disk scheduling — with definitions, formulas and solved numericals. Everything an SDE interview probes.

14	55+	10	8
SECTIONS	DEFINITIONS	SOLVED QS	FORMULA GROUPS

CONTENTS

01 Introduction to OS
02 OS Structure, Kernel & System Calls
03 Processes
04 Threads
05 CPU Scheduling
06 Process Synchronization
07 Classic Synchronization Problems
08 Deadlocks
09 Memory Management & Paging
10 Virtual Memory & Page Replacement
11 File Systems
12 Disk Scheduling
13 Practice Numericals (Solved)
14 Quick Revision Cheat-Sheet
15 Follow-up Drills

01 Introduction to OS

DEFINITION

An **Operating System (OS)** is system software that acts as an intermediary between the user and the computer hardware, managing resources (CPU, memory, I/O) and providing an environment in which programs run.

Functions of an OS

- **Process management** — creating, scheduling and terminating processes.
- **Memory management** — allocating and freeing memory, virtual memory.
- **File & I/O management** — organising files and controlling devices.
- **Security & protection** — isolating processes and controlling access.

TYPE	EXAMPLE / IDEA
Batch	Jobs grouped and run without interaction.
Time-sharing	Many users share the CPU via fast switching (Unix).
Real-time (RTOS)	Guaranteed response times (embedded, robotics).
Distributed	Manages a collection of networked machines.

02 OS Structure, Kernel & System Calls

DEFINITION — KERNEL

The **kernel** is the core part of the OS that always runs in **privileged (kernel) mode** and has direct access to hardware. It handles scheduling, memory, and system calls.

DEFINITION — SYSTEM CALL

A **system call** is the programmatic interface through which a user program requests a service from the kernel (e.g. `fork()`, `read()`, `open()`). It triggers a controlled switch from user mode to kernel mode.

	USER MODE	KERNEL MODE
Privilege	Restricted	Full hardware access
Runs	Applications	OS kernel
Switch via	System calls, interrupts, exceptions	

Monolithic kernel (Linux) = all services in kernel space (fast). **Microkernel** = minimal kernel, services in user space (safer, slower).

03 Processes

DEFINITION

A **process** is a program in execution — it includes the program code, the current activity (program counter, registers) and a **Process Control Block (PCB)** that stores all its metadata.

STATE	MEANING
New	Being created.
Ready	Waiting for the CPU.
Running	Currently executing on the CPU.
Waiting / Blocked	Waiting for I/O or an event.
Terminated	Finished execution.

Context switch — saving the state of one process and loading another's (overhead, no useful work). **PCB** stores PID, state, PC, registers, memory limits, open files.

04 Threads

DEFINITION

A **thread** is the smallest unit of execution within a process. Threads of the same process **share** code, data and files, but each has its own stack, registers and program counter.

	PROCESS	THREAD
Memory	Own address space	Shared within the process
Creation cost	High	Low (lightweight)
Context switch	Expensive	Cheaper
Communication	IPC needed	Direct (shared memory)

Multithreading models: many-to-one, one-to-one, many-to-many. **User threads** are managed in user space (fast, no kernel support); **kernel threads** are managed by the OS.

05 CPU Scheduling

DEFINITION

CPU scheduling is the OS activity of deciding which ready process gets the CPU next, to maximise utilisation and minimise waiting.

ALGORITHM	IDEA	ISSUE
FCFS	First come, first served	Convoy effect
SJF / SRTF	Shortest job (remaining) first	Starvation; needs burst estimates
Round Robin	Each gets a fixed time quantum	Quantum tuning
Priority	Highest priority first	Starvation (fixed by aging)

FORMULAS

```
Turnaround Time (TAT) = Completion − Arrival
Waiting Time (WT) = Turnaround − Burst
Response Time = First CPU − Arrival
```

Preemptive (SRTF, RR) can interrupt a running process; **non-preemptive** (FCFS, SJF) cannot.

06 Process Synchronization

DEFINITION

Process synchronization coordinates concurrent processes that share data so their interleaving produces a correct result. A **race condition** is when the outcome depends on the unpredictable order of access.

The **critical section** is the part of code that accesses shared data; only one process may execute it at a time. A correct solution needs **mutual exclusion**, **progress** and **bounded waiting**.

TOOL	MEANING
Mutex	A lock with lock/unlock (ownership).
Semaphore	An integer with `wait()`/`signal()`; counting or binary.
Monitor	A high-level construct with automatic mutual exclusion.

07 Classic Synchronization Problems

Producer–Consumer — Producer adds to a bounded buffer, consumer removes — needs empty/full semaphores + mutex.

Reader–Writer — Many readers or one writer; readers share, writers need exclusivity.

Dining Philosophers — Five philosophers, five forks; illustrates deadlock and starvation — solved by ordering forks or a waiter.

> WHY THEY MATTER
>
> These three problems cover almost every synchronisation question an interviewer asks — know the semaphore setup for each.

08 Deadlocks

> DEFINITION
>
> A **deadlock** is a state where a set of processes are blocked forever, each waiting for a resource held by another.

Four necessary conditions (all must hold): **Mutual exclusion**, **Hold and wait**, **No preemption**, **Circular wait**.

APPROACH	HOW
Prevention	Break one of the four conditions.
Avoidance	**Banker's algorithm** — grant a request only if the system stays in a safe state.
Detection & recovery	Build a resource-allocation graph, find a cycle, kill/rollback a process.
Ignore	The "ostrich" approach (most OSes).

BANKER'S ALGORITHM

Need = **Max − Allocation**
A state is **safe** if a sequence exists where every process's Need ≤ Work (available + released).

09 Memory Management & Paging

DEFINITION — PAGING

Paging divides physical memory into fixed-size **frames** and logical memory into equal-size **pages**; a **page table** maps each page to its frame, eliminating external fragmentation.

TECHNIQUE	IDEA
Contiguous	Each process in one block (first/best/worst fit) — fragmentation.
Paging	Fixed-size pages/frames — no external fragmentation.
Segmentation	Variable-size logical segments — matches program structure.

FORMULAS

Number of pages = **logical address space ÷ page size**
Offset bits = **$\log_2$(page size)**
Page table size = **number of entries × entry size**

EXAMPLE

32-bit address, page size 4 KB → offset = $\log_2(4096)$ = **12 bits**; pages = $2^{32} \div 2^{12} = 2^{20}$ = **1M pages**. With 4-byte entries the page table = 1M × 4 B = **4 MB**.

10 Virtual Memory & Page Replacement

DEFINITION

Virtual memory lets a process run even if it is not fully in RAM, keeping only the needed pages in memory and the rest on disk. A **page fault** occurs when a needed page is not in memory and must be fetched.

POLICY	RULE
FIFO	Evict the oldest loaded page. Can suffer **Belady's anomaly**.
LRU	Evict the least recently used page.
Optimal (OPT)	Evict the page used farthest in the future (theoretical best).

EFFECTIVE ACCESS TIME (WITH TLB)

$$EAT = h(t + m) + (1 - h)(t + 2m)$$

h = TLB hit ratio, t = TLB access, m = memory access

Thrashing — excessive paging that keeps the CPU busy swapping instead of working; fixed with the working-set model.

11 File Systems

DEFINITION

A **file system** is the part of the OS that organises and stores files on disk, providing a logical view (files, directories) over physical blocks.

ALLOCATION	IDEA	NOTE
Contiguous	File stored in consecutive blocks	Fast, but external fragmentation
Linked	Blocks linked by pointers	No fragmentation, slow random access
Indexed	An index block holds pointers	Fast random access (inode)

An **inode** stores a file's metadata (size, owner, permissions, block pointers) — but **not** its name. **Hard link** = another name for the same inode; **soft link** = a pointer to the path.

12 Disk Scheduling

DEFINITION

Disk scheduling decides the order in which pending disk I/O requests are served, to minimise total **head movement** (seek time).

ALGORITHM	IDEA
FCFS	Serve in arrival order (fair, slow).
SSTF	Serve the closest request (may starve far ones).
SCAN (elevator)	Sweep in one direction to the end, then reverse.
C-SCAN	Sweep one way, jump back, sweep again (uniform wait).
LOOK	Like SCAN but reverses at the last request, not the disk end.

METRIC

$$\text{Total head movement} = \sum |\text{current cylinder} - \text{next cylinder}|$$

13 Practice Numericals (Solved)

Q1 **FCFS: processes P1(AT 0, BT 5), P2(AT 1, BT 3), P3(AT 2, BT 1). Find average waiting and turnaround time.**

SOLUTION

- Order: P1 0–5, P2 5–8, P3 8–9
- TAT: P1=5, P2=7, P3=7 → avg = 19/3 ≈ **6.33**
- WT: P1=0, P2=4, P3=6 → avg = 10/3 ≈ **3.33**

Q2 **SJF (non-preemptive): same processes. Find average WT and TAT.**

SOLUTION

- t=0 only P1 → run P1 0–5; then P3 (BT 1) 5–6, then P2 6–9
- TAT: P1=5, P3=4, P2=8 → avg = 17/3 ≈ **5.67**
- WT: P1=0, P3=3, P2=5 → avg = 8/3 ≈ **2.67**

Q3 **Round Robin (quantum = 2) on the same processes. Find average WT.**

SOLUTION

- Gantt: P1(0–2) P2(2–4) P3(4–5) P1(5–7) P2(7–8) P1(8–9)
- CT: P1=9, P2=8, P3=5 → TAT = 9,7,3; WT = 4,4,2
- **Avg WT = 10/3 ≈ 3.33**

Q4 **Banker's: Available = [3,3,2]. Need = P0[7,4,3], P1[1,2,2], P2[6,0,0], P3[0,1,1], P4[4,3,1]. Find a safe sequence.**

SOLUTION

- P1 (1,2,2) ≤ (3,3,2) → Work = (5,3,2)
- P3 (0,1,1) ≤ (5,3,2) → Work = (7,4,3)
- P4 (4,3,1) ≤ (7,4,3) → Work = (7,4,5)
- P0 (7,4,3) ≤ (7,4,5) → Work = (7,5,5); P2 (6,0,0) ≤ (7,5,5)
- **Safe sequence: P1 → P3 → P4 → P0 → P2**

Q5 **FIFO page replacement: reference string 1,2,3,4,1,2,5,1,2,3,4,5 with 3 frames. Count page faults.**

SOLUTION

- 1 m → 2 m → 3 m → 4 m(evict 1) → 1 m(evict 2) → 2 m(evict 3) → 5 m(evict 4)
- 1 hit → 2 hit → 3 m(evict 1) → 4 m(evict 2) → 5 hit
- **Page faults = 9**

Q6 **LRU page replacement: same reference string, 3 frames. Count page faults.**

SOLUTION

- 1 m, 2 m, 3 m, 4 m(evict 1), 1 m(evict 2), 2 m(evict 3), 5 m(evict 4)
- 1 hit, 2 hit, 3 m(evict 5), 4 m(evict 1), 5 m(evict 2)
- **Page faults = 10 (Optimal on the same string gives 7)**

Q7 **A system has a 32-bit virtual address and a page size of 4 KB. Find the number of pages and the page-table size (4-byte entries).**

SOLUTION

- Offset bits = $\log_2(4096)$ = **12**
- Pages = $2^{32} \div 2^{12} = 2^{20}$ = **1,048,576 pages**
- Page table = $2^{20} \times 4$ B = **4 MB**

Q8 **TLB: hit ratio 0.9, TLB access 10 ns, memory access 100 ns. Find the effective access time.**

SOLUTION

- EAT = h(t+m) + (1−h)(t+2m)
- = 0.9(110) + 0.1(210) = 99 + 21
- **EAT = 120 ns**

Q9 **Disk (FCFS): queue 98,183,37,122,14,124,65,67; head at 53. Total head movement?**

SOLUTION

- 53 → 98(45), 98 → 183(85), 183 → 37(146), 37 → 122(85), 122 → 14(108), 14 → 124(110), 124 → 65(59), 65 → 67(2)
- **Total = 640 cylinders**

Q10 **Disk (SSTF): same queue and head. Total head movement?**

SOLUTION

- 53 → 65(12), 65 → 67(2), 67 → 37(30), 37 → 14(23), 14 → 98(84), 98 → 122(24), 122 → 124(2), 124 → 183(59)
- **Total = 236 cylinders (much better than FCFS)**

14 Quick Revision Cheat-Sheet

- OS manages CPU, memory, files, I/O; kernel runs in privileged mode.
- Process = program in execution + PCB; states: new, ready, running, waiting, terminated.
- Threads share memory, have their own stack/PC; cheaper to switch than processes.
- Scheduling: FCFS, SJF/SRTF, Round Robin, Priority. TAT = CT−AT; WT = TAT−BT.
- Sync: mutex, semaphore, monitor; classic problems = producer-consumer, reader-writer, dining philosophers.
- Deadlock: 4 conditions (mutual exclusion, hold&wait, no preemption, circular wait); Banker's avoids it.
- Paging removes external fragmentation; offset bits = $\log_2$(page size).
- Page replacement: FIFO (Belady's anomaly), LRU, Optimal. EAT = h(t+m)+(1−h)(t+2m).
- File allocation: contiguous, linked, indexed; inode stores metadata.
- Disk scheduling: FCFS, SSTF, SCAN, C-SCAN, LOOK.

MOST-ASKED

CPU scheduling numericals, **page replacement (count faults)**, **Banker's algorithm** and **deadlock conditions** — these dominate OS interviews.

15 Follow-up Drills

Interviewers rarely stop at the first answer. Each drill below is the main question, then the follow-ups they actually push on — practise the whole chain, not just the first line.

Q Process vs thread — what is the real difference?

↳ **What exactly is saved on a context switch?**

The CPU state: program counter, registers, stack pointer, and (for a process switch) the memory-management state such as page-table base. A thread switch saves less because the address space is shared.

↳ **Why do threads share the address space?**

That sharing is the point — threads communicate through shared memory without IPC, which is why they are cheaper than processes. The cost is that one thread can corrupt another's data.

Q What are the four conditions for a deadlock?

↳ **Prevention vs avoidance — what is the difference?**

Prevention breaks one of the four conditions structurally (for example, ordering resources so circular wait cannot form). Avoidance lets the conditions exist but checks each request against a safe state, which is what Banker's algorithm does.

↳ **Is Banker's algorithm used in practice?**

Rarely — it needs the maximum resource need of every process in advance, which real systems do not know. Most systems use detection plus recovery, or simply avoid the situation by design.

Q Why do we need virtual memory at all?

↳ **What is thrashing and how do you fix it?**

The system spends more time paging than executing because the working set does not fit in RAM. Fixes: reduce the multiprogramming degree, increase physical memory, or use a better page-replacement policy.

↳ **What does the TLB do?**

It caches recent virtual-to-physical translations so most accesses skip the page-table walk. A TLB miss costs extra memory reads, which is why a TLB flush on a process switch is expensive.

Q Compare FIFO, LRU and Optimal page replacement.

↳ Why is exact LRU hard to implement?

It needs a timestamp or ordering of every page on every access. Real systems approximate it with a clock/reference-bit scheme rather than tracking a true LRU order.

↳ What is Belady's anomaly?

With FIFO, increasing the number of frames can increase the number of page faults. LRU and Optimal are stack algorithms, so they never show this.

Q Which CPU scheduling algorithm suits an interactive system?

↳ What is the convoy effect?

One long CPU-bound process holds the CPU and many short I/O-bound processes queue behind it, hurting responsiveness. Round-robin or multilevel feedback queues prevent it.

↳ Why does round-robin need a good time quantum?

Too small and context-switch overhead dominates; too large and it degenerates into FCFS. It is a trade-off between responsiveness and throughput.

Q Mutex vs semaphore — when do you use which?

↳ Is a binary semaphore the same as a mutex?

Not quite. A mutex has ownership — only the thread that locked it can unlock it. A binary semaphore has no owner, so any thread can signal it, which makes it usable for signalling between threads.

↳ What is priority inversion and how do you fix it?

A high-priority task waits on a lock held by a low-priority task while a medium-priority task runs. Priority inheritance (the holder temporarily inherits the high priority) fixes it.

Q What happens inside the machine when a program makes a system call?

↳ Why can user code not just call the kernel function directly?

Because the kernel runs in a privileged mode with access to hardware and all memory. The CPU enforces the boundary; a trap switches to kernel mode, and the kernel validates arguments before acting.

↳ Why are system calls expensive compared with a function call?

They involve a mode switch, saving and restoring state, and losing some CPU cache locality. That is why buffered I/O and batching (fewer, larger syscalls) matter for performance.

Q What is an inode, and why does the file system use one?

> **↳ Hard link vs soft link — what is the difference?**
>
> A hard link is another directory entry pointing to the same inode, so the data is the same file with two names. A soft link is a separate file that stores a path, so it breaks if the target is deleted.

> **↳ Why is a directory lookup slower with millions of files?**
>
> Directory entries have to be searched (or hashed) to map a name to an inode; huge directories make that lookup expensive, which is why sharding files across subdirectories is common.